Evergreen STEM VirtualLab

Code Documentation

Unity 6 · 2D · Malay-language science virtual lab (experiments akt3-akt7, quizzes Uji Diri 1-7)

35 scripts documented · generated 2026-10-13 from the project source

Contents

1. Overview 4

2. Script Index 4

3. Script Reference 6

3.1 Navigation and Flow 6

SceneController.cs . . . 6

NextExperimentController.cs . . . 6

SceneName.cs . . . 7

DontDestroyCanvas.cs . . . 7

VersionNumberController.cs . . . 8

MenuSlideController.cs . . . 8

3.2 Quiz and Score 9

QuestionController.cs . . . 9

QuestionList.cs . . . 11

ScoreController.cs . . . 11

loadScore.cs . . . 12

3.3 Lab Interaction (drag, drop, triggers) 12

DragController.cs . . . 12

ItemSlot.cs . . . 13

triggerHit.cs . . . 14

newTrigger.cs . . . 15

newAfterTrigger.cs . . . 16

triggerAfter.cs . . . 16

TriggerController.cs . . . 17

Bikar7.cs . . . 18

CheckContent.cs . . . 19

CheckOnEnabled.cs . . . 19

ConditionalCheck.cs . . . 20

DisplayItem.cs . . . 20

ToDisplayContent.cs . . . 21

ObjectController.cs . . . 22

toggleController.cs . . . 22

FallingObject.cs . . . 23

3.4 Experiment-specific 23

WaterVolumeController.cs . . . 23

GetahCollection.cs . . . 24

SuhuBilikControl.cs . . . 25
TimerText.cs . . . 25
3.5 Visual and Audio **26**
WaterShapeController.cs . . . 26
WaterSpring.cs . . . 27
MusicLoopingController.cs . . . 28
TextSlider.cs . . . 29
transformObject.cs . . . 29
4. Appendix - Full Source of the Largest Scripts **31**
triggerHit.cs . . . 31
ItemSlot.cs . . . 35
QuestionController.cs . . . 38

1. Overview

• Evergreen STEM VirtualLab is a 2D Unity game (Unity 6) with Malay-language science (KPS) experiments (akt3-akt7) and quizzes (Uji Diri 1-7).

• Flow: LoginPage -> Dashboard -> an experiment or quiz scene -> back to Dashboard (NextExperimentController.tamat).

• Experiments are built from step panels. Each step has a ConditionalCheck that decides when the 'next' button appears. Items are moved with DragController and placed or detected by ItemSlot and the trigger scripts.

• Quizzes read their data from QuestionList, are run by QuestionController and score with ScoreController.

• Scripts are MonoBehaviours kept flat in Assets/. Older files use camelCase names (triggerHit, newTrigger, loadScore) and must not be renamed because scene references depend on them.

• Third-party code (UnityMCP, OpenAI-Unity chat packages, TextMesh Pro) is not covered; the project contains no custom AI-chat scripts of its own.

Note: the 'Used in' column is derived from scene names referenced in the code and from the project docs; scene attachments were not inspected in the Unity Editor.

2. Script Index

1. Navigation and Flow

Moves the player between scenes and between steps inside a scene.

Script	What it does	Used in
SceneController.cs	Small helper wired to UI buttons to load, reload or quit.	Dashboard, quiz and experiment buttons
NextExperimentController.cs	Put on the 'Next' button of a step panel. Hides the current step panel and shows the next sibling panel.	Experiments (akt3-akt7)
SceneName.cs	Writes the active scene's name into the TextMeshPro label it is attached to.	Any scene with a title label
DontDestroyCanvas.cs	Singleton that keeps its canvas alive across scene loads.	Persistent AI-assistant canvas
VersionNumberController.cs	Shows Application.version (Player Settings version) in a TextMeshPro label.	LoginPage / Dashboard footer
MenuSlideController.cs	Vertical paging through child panels with up and down arrow buttons.	Dashboard / menu panels

2. Quiz and Score

Runs the 'Uji Diri' self-test quizzes and shows the score.

Script	What it does	Used in
QuestionController.cs	Main quiz engine: displays the question, handles answer feedback, awards marks and advances questions.	Quiz scenes 'Uji Diri 1-7'
QuestionList.cs	Pure data holder for quiz content (questions and answers), editable in the Inspector.	Quiz scenes
ScoreController.cs	Singleton that stores the player's score and writes it into a TextMeshPro label.	Quiz scenes
loadScore.cs	Displays the final result as 'score / maxScore' where maxScore = number of questions x 10.	Quiz result screen

3. Lab Interaction (drag, drop, triggers)

The core experiment mechanics: dragging lab items and checking procedure steps.

Script	What it does	Used in
DragController.cs	Makes a UI item draggable on the Canvas (drag begin, drag, drag end).	All experiments
ItemSlot.cs	A drop target (slot). Snaps the dropped item into place and shows a tick; in akt6/akt7 also accepts electrodes/wires by trigger.	akt5, akt6, akt7 (drop targets)

Script	What it does	Used in
triggerHit.cs	Largest script. Per-object trigger logic for akt3 procedures 2-14 (flame, culture solution, zigzag streaking, incubation) plus one akt7 step.	akt3 (microbiology), akt7
newTrigger.cs	Trigger zone that reveals follow-up objects when an item tagged 'action' enters it.	Experiments with 'action' tagged items
newAfterTrigger.cs	Like newTrigger but fires on exit: when an 'action' item leaves, activates itemtodo and locks the item.	Experiments
triggerAfter.cs	Trigger in akt4 that gradually shows an item (CanvasGroup alpha +0.09 per hit) with a percentage label.	akt4
TriggerController.cs	Sequencer for akt4 step 3 and step 6 (repeated pipetting rounds).	akt4
Bikar7.cs	Beaker trigger for akt7: every 'action' item entering fills the beaker image by 20%; on exit it enables the next button and the stirrer and locks the item.	akt7 (beaker)
CheckContent.cs	Checks (every frame) whether items in a list are active/inactive and then shows up to three objects.	Experiments
CheckOnEnabled.cs	On Start calls ConditionalCheck.CheckCondition() so a step is evaluated as soon as its panel appears.	Experiments
ConditionalCheck.cs	Procedure-step checker. Decides when the 'next' button and the 'correct' marker are shown.	All experiments (one per procedure panel)
DisplayItem.cs	SetupWire() activates all items in a list and then asks ConditionalCheck to evaluate the step.	Experiments (wire/connection steps)
ToDisplayContent.cs	Every frame, when all items in 'items' are inactive, activates all items in 'toDisplayItem'.	Experiments
ObjectController.cs	HideObject() deactivates its own GameObject; EnableObject() activates a referenced object. Meant for UnityEvents on buttons.	Experiments (button events)
toggleController.cs	toggleOn() flips the visibility of object1 on each click.	Experiments (toggle buttons)
FallingObject.cs	Gives a Rigidbody2D a downward velocity and lets the arrow keys steer it; used to drop objects into the water.	Water simulation demo

4. Experiment-specific

Scripts that belong to a single experiment or to timing/temperature displays.

Script	What it does	Used in
WaterVolumeController.cs	AddLevel() activates the first inactive water-level object in the list, raising the visible liquid level one step.	akt4
GetahCollection.cs	Steps through a list of collection items one at a time.	akt7 (latex / getah collection)
SuhuBilikControl.cs	Drives the temperature slider while a flame ('api') heats an item during procedures 4 and 11.	akt3 (temperature control)
TimerText.cs	Accelerated clock or warming thermometer shown in a TextMeshPro label.	Experiments with a clock or thermometer

5. Visual and Audio

Water simulation, music and small UI helpers.

Script	What it does	Used in
WaterShapeController.cs	Generates and animates a wavy water surface on a SpriteShapeController ([ExecuteAlways]).	Water scenes (SpriteShape)
WaterSpring.cs	One point on the water surface, simulated as a damped spring (Hooke's law with dampening).	Water scenes
MusicLoopingController.cs	Persistent singleton that plays a playlist of AudioSources one after another and loops back to the first.	All scenes (persistent audio)
TextSlider.cs	Finds the child named 'text' and exposes it as a TextMeshProUGUI so other scripts can show the slider value.	Sliders with a value label
transformObject.cs	DisableObject() deactivates its GameObject; called from animation events when a clip ends.	Animation events

3. Script Reference

3.1 Navigation and Flow

Moves the player between scenes and between steps inside a scene.

SceneController.cs

Assets/SceneController.cs · 28 lines · used in: Dashboard, quiz and experiment buttons

Small helper wired to UI buttons to load, reload or quit.

- ChangeScene(string) loads a scene asynchronously by name (e.g. 'akt3', 'Uji Diri 1').
- ReloadScene() reloads the active scene; QuitGame() closes the application.
- Start/Update are empty template methods.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class SceneController : MonoBehaviour
{
    public void QuitGame()
    {
        Application.Quit();
    }

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update() { }

    public void ReloadScene()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }

    public void ChangeScene(string sceneName)
    {
        SceneManager.LoadSceneAsync(sceneName);
    }
}
```

Full file (28 lines)

NextExperimentController.cs

Assets/NextExperimentController.cs · 56 lines · used in: Experiments (akt3-akt7)

Put on the 'Next' button of a step panel. Hides the current step panel and shows the next sibling panel.

- On Start it finds the next sibling of its parent panel and registers the button listener.
- If the current panel is the last one, the button label becomes 'TAMAT' and clicking it calls tamat(), which loads the 'Dashboard' scene.
- Otherwise NextExperiment() deactivates the parent and activates nextParent.
- Relies on the hierarchy: the step panels must be siblings in the correct order.

```
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

public class NextExperimentController : MonoBehaviour
{
    [SerializeField]
    private Transform parent;

    [SerializeField]
    private Transform nextParent;

    // Start is called before the first frame update
    void Start()
    {
        parent = transform.parent;
        if (transform.parent.GetSiblingIndex() + 1 != transform.parent.transform.parent.childCount)
        {
            for (int i = 0; i < transform.parent.transform.parent.childCount; i++)
            {
                if (parent.transform.parent.GetChild(i).gameObject.activeSelf)
                {
                    nextParent = parent.transform.parent.GetChild(i + 1).transform;
                }
            }
        }
        if (transform.parent.GetSiblingIndex() + 1 == transform.parent.transform.parent.childCount)
        {
            GetComponent<Button>().transform.GetChild(0).GetComponent<TextMeshProUGUI>().text =
                "TAMAT";

            GetComponent<Button>().onClick.AddListener(() => tamat());
        }
        else
        {
            GetComponent<Button>().onClick.AddListener(() => NextExperiment());
        }
    }

    public void tamat()
    {
        SceneManager.LoadScene("Dashboard");
    }

    public void NextExperiment()
    {
        parent.gameObject.SetActive(false);
        nextParent.gameObject.SetActive(true);
    }

    // Update is called once per frame
    void Update() { }
}
```

Full file (56 lines)

SceneName.cs

Assets/SceneName.cs · 17 lines · used in: Any scene with a title label

Writes the active scene's name into the TextMeshPro label it is attached to.

```
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;

public class SceneName : MonoBehaviour
{
    // Start is called before the first frame update
    void Start()
    {
        GetComponent<TextMeshProUGUI>().text = SceneManager.GetActiveScene().name;
    }

    // Update is called once per frame
    void Update() { }
}
```

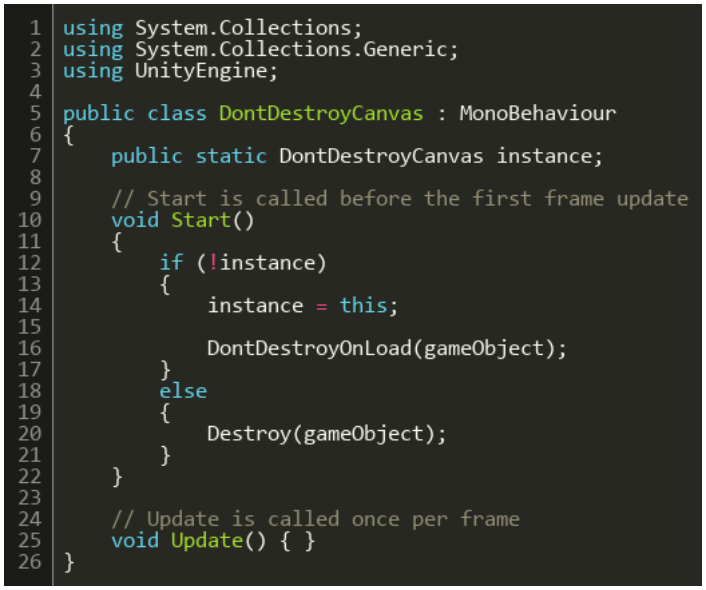
Full file (17 lines)

DontDestroyCanvas.cs

Assets/Scenes/DontDestroyCanvas.cs · 26 lines · used in: Persistent AI-assistant canvas

Singleton that keeps its canvas alive across scene loads.

- First instance stores itself in 'instance' and calls DontDestroyOnLoad; later duplicates destroy themselves.
- Used for the persistent chat canvas so the assistant survives scene changes.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DontDestroyCanvas : MonoBehaviour
{
    public static DontDestroyCanvas instance;

    // Start is called before the first frame update
    void Start()
    {
        if (!instance)
        {
            instance = this;

            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
    }

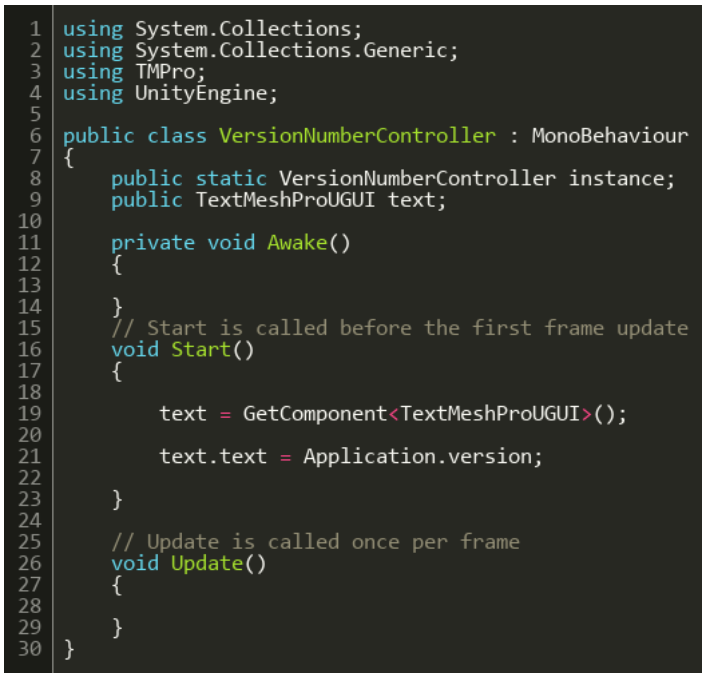
    // Update is called once per frame
    void Update() { }
}
```

Full file (26 lines)

VersionNumberController.cs

Assets/VersionNumberController.cs · 30 lines · used in: LoginPage / Dashboard footer

Shows Application.version (Player Settings version) in a TextMeshPro label.



```
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;

public class VersionNumberController : MonoBehaviour
{
    public static VersionNumberController instance;
    public TextMeshProUGUI text;

    private void Awake()
    {

    }
    // Start is called before the first frame update
    void Start()
    {

        text = GetComponent<TextMeshProUGUI>();

        text.text = Application.version;

    }

    // Update is called once per frame
    void Update()
    {

    }
}
```

Full file (30 lines)

MenuSlideController.cs

Assets/MenuSlideController.cs · 72 lines · used in: Dashboard / menu panels

Vertical paging through child panels with up and down arrow buttons.

- Start caches all children into Contents.
- UpArrow()/DownArrow() deactivate the active child and activate the previous/next one, clamped to the first/last index.
- Contains Debug.Log calls that could be removed.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class MenuSlideController : MonoBehaviour
{
    public Transform currentTransform;
    public int index;

    public List<Transform> Contents;
    // Start is called before the first frame update
    void Start()
    {
        for (int i = 0; i < transform.childCount; i++)
        {
            Contents.Add(transform.GetChild(i));
        }
    }

    // Update is called once per frame
    void Update()
    {

    }


    public void UpArrow()
    {
        for (int i = 0; i < transform.childCount; i++)
        {
            if (transform.GetChild(i).gameObject.activeSelf)
            {
                Debug.Log(transform.childCount);
                currentTransform = transform.GetChild(i).transform;
                currentTransform.gameObject.SetActive(false);
                index = currentTransform.transform.GetSiblingIndex();
                index--;

                if (index < 0)
                {
                    index = 0;
                }
                Contents[index].transform.gameObject.SetActive(true);
                return;
            }
        }

    }
    public void DownArrow()
    {
        for (int i = 0; i < transform.childCount; i++)
        {
            if (transform.GetChild(i).gameObject.activeSelf)
            {
                Debug.Log(transform.childCount);
                currentTransform = transform.GetChild(i).transform;
                currentTransform.gameObject.SetActive(false);
                index = currentTransform.transform.GetSiblingIndex();
                index++;

                if (index >= transform.childCount)
                {

                    index = transform.childCount - 1;
                }
                Contents[index].transform.gameObject.SetActive(true);
                return;
            }
        }

    }
}
```

Full file (72 lines)

3.2 Quiz and Score

Runs the 'Uji Diri' self-test quizzes and shows the score.

QuestionController.cs

Assets/QuestionController.cs · 277 lines · used in: Quiz scenes 'Uji Diri 1-7'

Main quiz engine: displays the question, handles answer feedback, awards marks and advances questions.

- Start() resets counters and shows question 0 using data from QuestionList.instance.
- AddMark() (correct) and IncorrectButton() (wrong) disable all answer buttons, then run coroutines that play sounds ('yay', 'claps', 'salah') and change the avatar face.
- addingmark() waits 1.3 s then adds 10 points to ScoreController and refreshes the label.
- NextStepping() -> nextStep(): highlights the real answer (green for correct, red otherwise), moves to the next question, or shows CongratPrompt after the last one.
- Uses GameObject.Find for audio sources, so the objects 'yay', 'claps' and 'salah' must exist in each quiz scene.
- startQuestion() is an empty leftover.

```
186     public void AddMark()
187     {
188         //turn everybutton to disable
189         disableAllButtons(false);
190 
191         StartCoroutine(addingmark());
192     }
193 
194     public void IncorrectButton()
195     { //turn everybutton to disable
196         disableAllButtons(false);
197 
198         StartCoroutine(incorrect());
199     }
200 
201     IEnumerator incorrect()
202     {
203         yield return new WaitForSeconds(1);
204         if (!GameObject.Find("salah").GetComponent<AudioSource>().isPlaying)
205         {
206             GameObject.Find("salah").GetComponent<AudioSource>().Play();
207         }
208         //sad
209         AvatarSad();
210     }
211 
212     IEnumerator addingmark()
213     {
214         yield return new WaitForSeconds(1);
215         if (!GameObject.Find("yay").GetComponent<AudioSource>().isPlaying)
216         {
217             GameObject.Find("yay").GetComponent<AudioSource>().Play();
218         }
219         if (!GameObject.Find("claps").GetComponent<AudioSource>().isPlaying)
220         {
221             GameObject.Find("claps").GetComponent<AudioSource>().Play();
222         }
223 
224         //smile
225         AvatarSmile();
226 
227         //play animation add coin and change text
228         //        GameObject.Find("Coin").GetComponent<Animator>().Play("addCoin");
229         yield return new WaitForSeconds(1.3f);
230         ScoreController.instance.score += 10;
231         ScoreController.instance.SetScore();
232     }
```

Lines 186-233 of 277

```
234     IEnumerator nextStep()
235     {
236         //turn everybutton to red
237         for (int i = 0; i < answerBtn.Count; i++)
238         {
239             answerBtn[i].GetComponent<Button>().enabled = false;
240         }
241         yield return new WaitForSeconds(1.2f);
242         showRealAnswer();
243         countStep++;
244         yield return new WaitForSeconds(3);
245         countQuestion++;
246         if (countQuestion >= QuestionList.instance.questionlist.Count)
247         {
248             //countStep++;
249             // applyQuestion();
250             //  applyAnswerOnButton();
251             CongratPrompt.gameObject.SetActive(true);
252         }
253         else
254         {
255             countStep = 0;
256             applyQuestion();
257             applyAnswerOnButton();
258         }
259 
260         disableAllButtons(true);
261         resetButton();
262         defaultAvatarFace();
263     }
264 
265     public void NextStepping()
266     {
267         StartCoroutine(nextStep());
268     }
```

Lines 234-269 of 277

QuestionList.cs

Assets/QuestionList.cs · 37 lines · used in: Quiz scenes

Pure data holder for quiz content (questions and answers), editable in the Inspector.

- QuestionAndAnswer: a list of question strings plus a list of AnswerOnButton.
- AnswerOnButton: the answer texts and the index (answeOnBtn) of the correct button.
- Exposes a static 'instance' set in Awake for QuestionController and loadScore.

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class QuestionList : MonoBehaviour
{
    public static QuestionList instance;

    [SerializeField]
    public List<QuestionAndAnswer> questionlist = new List<QuestionAndAnswer>();

    void Awake()
    {
        instance = this;
    }

    [System.Serializable]
    public class QuestionAndAnswer
    {
        public List<String> question;
        public List<AnswerOnButton> QuestionOnAnswersBtn = new List<AnswerOnButton>();
    }

    [System.Serializable]
    public class AnswerOnButton
    {
        public List<string> answer;
        public int answeOnBtn;
    }

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update() { }
}
```

Full file (37 lines)

ScoreController.cs

Assets/ScoreController.cs · 29 lines · used in: Quiz scenes

Singleton that stores the player's score and writes it into a TextMeshPro label.

- score is incremented by QuestionController (10 per correct answer).
- SetScore() refreshes textScore.

```cs
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;

public class ScoreController : MonoBehaviour
{
    public static ScoreController instance;

    public int score;

    public TextMeshProUGUI textScore;

    void Awake()
    {
        instance = this;
    }

    public void SetScore()
    {
        textScore.text = score.ToString();
    }

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update() { }
}
```

Full file (29 lines)

loadScore.cs

Assets/loadScore.cs · 18 lines · used in: Quiz result screen

Displays the final result as 'score / maxScore' where maxScore = number of questions x 10.

```cs
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;

public class loadScore : MonoBehaviour
{
    // Start is called before the first frame update
    void Start()
    {
        // 10 points per question (see QuestionController.addingmark)
        int maxScore = QuestionList.instance.questionlist.Count * 10;
        GetComponent<TextMeshProUGUI>().text = ScoreController.instance.score + " / " + maxScore;
    }

    // Update is called once per frame
    void Update() { }
}
```

Full file (18 lines)

3.3 Lab Interaction (drag, drop, triggers)

The core experiment mechanics: dragging lab items and checking procedure steps.

DragController.cs

Assets/DragController.cs · 81 lines · used in: All experiments

Makes a UI item draggable on the Canvas (drag begin, drag, drag end).

- OnDrag moves anchoredPosition by the pointer delta divided by canvas scale.
- OnBeginDrag adjusts opacity, raycast blocking and collider depending on objectName ('Dawai', 'katod', 'getahasli'); 'getahasli' also tilts the item by -32 degrees.
- OnEndDrag restores rotation (except in akt4), alpha, raycasts and collider.
- Other scripts disable this component to 'lock' an item once it has been placed correctly.
- Start finds the Canvas by name ('Canvas').

```
7  public class DragController : MonoBehaviour, IDragHandler, IEndDragHandler, IBeginDragHandler
8  {
9      [SerializeField]
10     private string objectName;
11
12     [SerializeField]
13     private Canvas canvas;
14
15     public void OnBeginDrag(PointerEventData eventData)
16     {
17         if (objectName == null)
18         {
19             //  Debug.Log("OnBeginDrag");
20             GetComponent<CanvasGroup>().alpha = .6f;
21             GetComponent<CanvasGroup>().blocksRaycasts = false;
22             //        GameObject.Find("cartoon").GetComponent<AudioSource>().Play();
23             GetComponent<BoxCollider2D>().enabled = false;
24         }
25         else if (objectName.Equals("Dawai"))
26         {
27             //  Debug.Log("OnBeginDrag");
28             GetComponent<CanvasGroup>().alpha = 1f;
29             GetComponent<CanvasGroup>().blocksRaycasts = true;
30             //        GameObject.Find("cartoon").GetComponent<AudioSource>().Play();
31             GetComponent<BoxCollider2D>().enabled = true;
32         }
33         else if (objectName.Equals("katod"))
34         {
35             //  Debug.Log("OnBeginDrag");
36             GetComponent<CanvasGroup>().alpha = 1f;
37             GetComponent<CanvasGroup>().blocksRaycasts = true;
38             //        GameObject.Find("cartoon").GetComponent<AudioSource>().Play();
39             GetComponent<BoxCollider2D>().enabled = true;
40         }
41         else if (objectName.Equals("getahasli"))
42         {
43             Debug.Log("drag");
44             GetComponent<RectTransform>().localRotation = Quaternion.Euler(0, 0, -32f);
45             ;
46             //  Debug.Log("OnBeginDrag");
47             GetComponent<CanvasGroup>().alpha = .8f;
48             GetComponent<CanvasGroup>().blocksRaycasts = true;
49             //        GameObject.Find("cartoon").GetComponent<AudioSource>().Play();
50             GetComponent<BoxCollider2D>().enabled = true;
51         }
52     }
53
54     public void OnDrag(PointerEventData eventData)
55     {
56         //Debug.Log("OnDrag");
57         GetComponent<RectTransform>().anchoredPosition += eventData.delta / canvas.scaleFactor;
58     }
59
60     public void OnEndDrag(PointerEventData eventData)
61     {
62         //   Debug.Log("OnEndDrag");
63         if (SceneManager.GetActiveScene().name != "akt4")
64         {
65             GetComponent<RectTransform>().localRotation = Quaternion.Euler(0, 0, 0);
66         }
```

Lines 7-66 of 81

ItemSlot.cs

Assets/ItemSlot.cs · 308 lines · used in: akt5, akt6, akt7 (drop targets)

A drop target (slot). Snaps the dropped item into place and shows a tick; in akt6/akt7 also accepts electrodes/wires by trigger.

- OnDrop (IDropHandler): re-parents the dragged item to the slot, centres it, shows the tick, disables the item's DragController and hides the slot image. akt5 offsets the item by 100 px; akt6 does nothing on drop.
- OnTriggerStay2D handles akt6/akt7 steps by matching the dragged object's name (rubberfilm, anodElektrod, katodElektrod, wayarAnod, ...) with the slot type (jenisAnodKatod) and the current procedure panel name.
- Many near-identical blocks per procedure; a good candidate for refactoring into a lookup table.

```
29     public void OnDrop(PointerEventData eventData)
30     {
31         nameScene = SceneManager.GetActiveScene().name.ToString();
32         Debug.Log("onDrop");
33         if (!SceneManager.GetActiveScene().name.Equals("akt6"))
34         {
35             //Debug.Log("stuck");
36             eventData.pointerDrag.transform.SetParent(transform);
37             //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition = GetComponent<RectTransform>().anchoredPosition;
38             eventData.pointerDrag.transform.localPosition = new Vector3(0, 0, 0);
39
40             tick.gameObject.SetActive(true);
41
42             eventData.pointerDrag.transform.GetComponent<DragController>().enabled = false;
43
44             if (SceneManager.GetActiveScene().name.Equals("akt5"))
45             {
46                 eventData.pointerDrag.transform.localPosition = new Vector3(0, 100f, 0);
47             }
48         }
49         else
50         {
51             Debug.Log("stuck");
52         }
53         GetComponent<Image>().enabled = false;
54     }
55
56     void OnTriggerStay2D(Collider2D collision)
```

Lines 29-56 of 308

```
56      void OnTriggerStay2D(Collider2D collision)
57      {
58          if (SceneManager.GetActiveScene().name == "akt7")
59          {
60              if (ConditionalCheck.name == "3")
61              {
62                  if (jenisAnodKatod == "anod")
63                  {
64                      Debug.Log("anod");
65                      if (collision.name.Equals("rubberfilm"))
66                      {
67                          collision.transform.GetComponent<DragController>().enabled = false;
68                          //Debug.Log("stuck");
69                          collision.transform.SetParent(transform);
70                          //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition = GetComponent<RectTransform>().anchoredPosition;
71                          collision.transform.localPosition = new Vector3(0, 0, 0);
72
73                          tick.gameObject.SetActive(true);
74                          ButtonNext.gameObject.SetActive(true);
75                          //transform.gameObject.SetActive(false);
76                      }
77                  }
78              }
79              if (ConditionalCheck.name == "2")
80              {
81                  if (jenisAnodKatod == "anod")
82                  {
83                      Debug.Log("anod");
84                      if (collision.name.Equals("rubberfilm"))
85                      {
```

Lines 56-85 of 308

triggerHit.cs

Assets/triggerHit.cs · 388 lines · used in: akt3 (microbiology), akt7

Largest script. Per-object trigger logic for akt3 procedures 2-14 (flame, culture solution, zigzag streaking, incubation) plus one akt7 step.

- Start caches objectName (the GameObject name) and the cooldown timer base (tempTime).
- OnTriggerEnter2D / OnTriggerStay2D / OnTriggerExit2D branch on the scene name, the current procedure number (ConditionalCheck.procedurNum) and objectName ('api' = flame, 'larutanKultur', 'collider', 'circle').
- Drives the TempSlider value (i) and text, fades the Petri dish (CanvasGroup alpha) and calls conditionalCheck.CheckCondition() when a step completes.
- Procedur13() raises the temperature slider by 10 per call and fades the Petri dish at thresholds (>40 half, >60 full).
- Uses GameObject.Find and long Transform.parent chains, so it is tightly coupled to the scene hierarchy.

```
34      public void Procedur13()
35      {
36          GameObject.Find("TempSlider").GetComponent<Slider>().value = i;
37          i += 10;
38          //Debug.Log("apiTrigger");
39
40          if (i <= GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
41          {
42              if (i > 40 && i < 60)
43              {
44                  GameObject
45                      .Find("Petri")
46                      .GetComponent<Transform>()
47                      .GetChild(0)
48                      .GetComponent<CanvasGroup>()
49                      .alpha = 0.5f;
50              }
51              else if (i > 60)
52              {
53                  GameObject
54                      .Find("Petri")
55                      .GetComponent<Transform>()
56                      .GetChild(0)
57                      .GetComponent<CanvasGroup>()
58                      .alpha = 1f;
59              }
60              GameObject.Find("TempSlider").GetComponent<TextSlider>().text.text = i.ToString();
61              if (i == GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
62              {
63                  isCooldown = false;
64
65                  conditionalCheck.CheckCondition();
66              }
67          }
68      }
```

Lines 34-69 of 388

```
70      void OnTriggerEnter2D(Collider2D other)
71      {
72          if (SceneManager.GetActiveScene().name.Equals("akt3"))
73          {
74              if (conditionalCheck.procedurNum == 2)
75              {
76                  if (objectName.Equals("api"))
77                  {
78                      transform
79                          .parent.transform.parent.transform.GetChild(1)
80                          .gameObject.SetActive(true);
81                      //Debug.Log("onEnter");
82                  }
83              }
84
85              if (conditionalCheck.procedurNum == 5)
86              {
87                  if (objectName.Equals("larutanKultur"))
88                  {
89                      transform
90                          .parent.transform.parent.transform.GetChild(1)
91                          .gameObject.SetActive(true);
92                      //Debug.Log("onEnter");
93                  }
94              }
95              if (conditionalCheck.procedurNum == 6)
96              {
97                  if (objectName.Equals("collider"))
```

Lines 70-97 of 388

newTrigger.cs

Assets/newTrigger.cs · 52 lines · used in: Experiments with 'action' tagged items

Trigger zone that reveals follow-up objects when an item tagged 'action' enters it.

- If isAdding is false: activates every object in itemtodo and disables the item's DragController.
- If isAdding is true: fills child 3's Image by 0.2 per hit; when full, plays the 'dropletStart' animation and activates itemtodo.

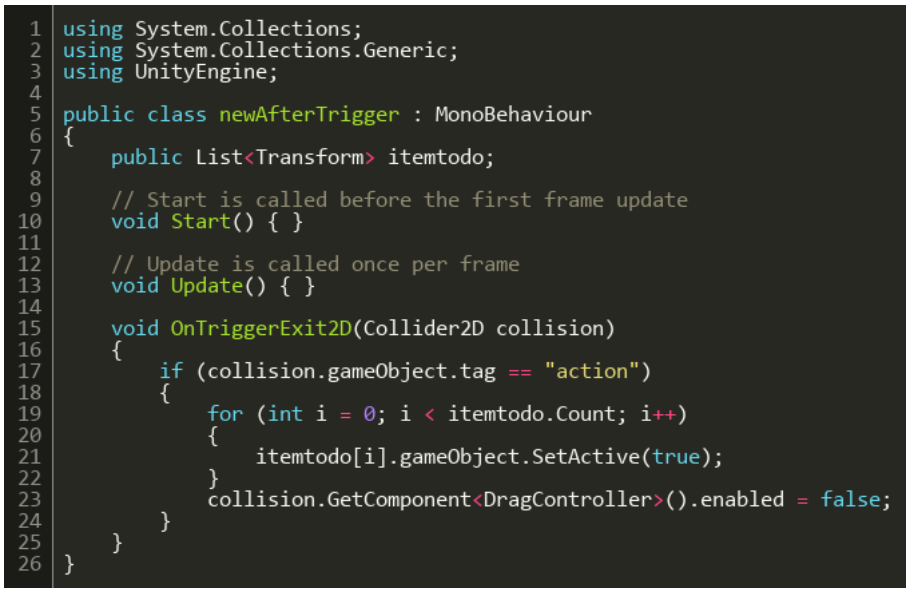
```
1   using System.Collections;
2   using System.Collections.Generic;
3   using UnityEngine;
4   using UnityEngine.UI;
5
6   public class newTrigger : MonoBehaviour
7   {
8       public List<Transform> itemtodo;
9
10      public bool isAdding;
11
12      // Start is called before the first frame update
13      void Start() { }
14
15      // Update is called once per frame
16      void Update() { }
17
18      void OnTriggerEnter2D(Collider2D collision)
19      {
20          if (collision.gameObject.tag == "action")
21          {
22              if (!isAdding)
23              {
24                  for (int i = 0; i < itemtodo.Count; i++)
25                  {
26                      itemtodo[i].gameObject.SetActive(true);
27                  }
28                  collision.GetComponent<DragController>().enabled = false;
29              }
30              else
31              {
32                  if (transform.GetChild(3).GetComponent<Image>().fillAmount != 1)
33                  {
34                      Debug.Log(transform.name);
35                      transform.GetChild(3).GetComponent<Image>().fillAmount += 0.2f;
36                  }
37                  else
38                  {
39                      if (GetComponent<Animator>())
40                      {
41                          GetComponent<Animator>().Play("dropletStart");
42                      }
43                      for (int i = 0; i < itemtodo.Count; i++)
44                      {
45                          itemtodo[i].gameObject.SetActive(true);
46                      }
47                      collision.transform.GetComponent<DragController>().enabled = false;
48                  }
49              }
50          }
51      }
52  }
```

Full file (52 lines)

newAfterTrigger.cs

Assets/newAfterTrigger.cs · 26 lines · used in: Experiments

Like newTrigger but fires on exit: when an 'action' item leaves, activates itemtodo and locks the item.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class newAfterTrigger : MonoBehaviour
{
    public List<Transform> itemtodo;

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update() { }

    void OnTriggerExit2D(Collider2D collision)
    {
        if (collision.gameObject.tag == "action")
        {
            for (int i = 0; i < itemtodo.Count; i++)
            {
                itemtodo[i].gameObject.SetActive(true);
            }
            collision.GetComponent<DragController>().enabled = false;
        }
    }
}
```

Full file (26 lines)

triggerAfter.cs

Assets/triggerAfter.cs · 66 lines · used in: akt4

Trigger in akt4 that gradually shows an item (CanvasGroup alpha +0.09 per hit) with a percentage label.

- At full alpha it straightens the item, shows actionButtons and hides thingsToRemove.
- When the colliding object is 'plasmid' or 'plasmid2' it hides itself and calls TriggerController.CheckItemAct4_3 or CheckItemAct4_6.



```
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

public class triggerAfter : MonoBehaviour
{
    public CanvasGroup itemToAppear;
    public TextMeshProUGUI textPercentage;

    public List<Transform> actionButtons;

    public List<Transform> thingsToRemove;

    [Space]
    public TriggerController triggerController;

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update() { }

    void OnTriggerEnter2D(Collider2D collision)
    {
        if (SceneManager.GetActiveScene().name == "akt4")
        {
            if (collision.gameObject.tag == "action")
            {
                itemToAppear.alpha += 0.09f;
                textPercentage.text = (itemToAppear.alpha * 100).ToString("00") + "%";
                Debug.Log("ItemTouch");

                if (itemToAppear.alpha == 1)
                {
                    collision.GetComponent<RectTransform>().localRotation = Quaternion.Euler(
                        0,
                        0,
                        0
                    );
                    for (int i = 0; i < actionButtons.Count; i++)
                    {
                        actionButtons[i].gameObject.SetActive(true);
                    }
                    for (int i = 0; i < thingsToRemove.Count; i++)
                    {
                        thingsToRemove[i].gameObject.SetActive(false);
                    }
                }
            }

            if (collision.gameObject.name == "plasmid")
            {
                transform.gameObject.SetActive(false);
                triggerController.CheckItemAct4_3();
            }
            else if (collision.gameObject.name == "plasmid2")
            {
                transform.gameObject.SetActive(false);
                triggerController.CheckItemAct4_6();
            }
        }
    }
}
```

Full file (66 lines)

TriggerController.cs

Assets/TriggerController.cs · 110 lines · used in: akt4

Sequencer for akt4 step 3 and step 6 (repeated pipetting rounds).

- CheckItemAct4_3 / CheckItemAct4_6 start coroutines that check whether all toHide items are inactive.
- If so, the current toDisplay item is hidden, turnCount increases and the next one is shown while WaterVolumeController(s) raise the water level.
- After the last turn (4 in step 3, 5 in step 6) the continue buttons appear.

```
40      IEnumerator checkItemAct4_6()
41      {
42          for (int i = 0; i < toHide.Count; i++)
43          {
44              if (toHide[i].gameObject.activeSelf)
45              {
46                  AllInactive = false;
47              }
48              else
49              {
50                  AllInactive = true;
51              }
52          }
53
54          if (AllInactive)
55          {
56              toDisplay[turnCount].gameObject.SetActive(false);
57              turnCount++;
58              if (turnCount < 5)
59              {
60                  toDisplay[turnCount].gameObject.SetActive(true);
61                  waterVolumeController2[0].AddLevel();
62
63                  yield return new WaitForSeconds(1.4f);
64                  //redisplay to hide
65                  toHide[0].gameObject.SetActive(true);
66              }
67              else
68              {
69                  buttonsContinue2[0].gameObject.SetActive(true);
70                  buttonsContinue2[1].gameObject.SetActive(true);
71              }
72          }
73      }
```

Lines 40-73 of 110

Bikar7.cs

Assets/Bikar7.cs · 38 lines · used in: akt7 (beaker)

Beaker trigger for akt7: every 'action' item entering fills the beaker image by 20%; on exit it enables the next button and the stirrer and locks the item.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.UI;
5
6  public class Bikar7 : MonoBehaviour
7  {
8      public Image fill;
9
10     public Transform stirrer;
11
12     // Start is called before the first frame update
13     void Start() { }
14
15     // Update is called once per frame
16     void Update() { }
17
18     void OnTriggerEnter2D(Collider2D collision)
19     {
20         Debug.Log("trigger");
21         if (collision.gameObject.tag == "action")
22         {
23             fill.fillAmount += 0.2f;
24         }
25     }
26
27     void OnTriggerExit2D(Collider2D collision)
28     {
29         if (collision.gameObject.tag == "action")
30         {
31             collision
32                 .transform.parent.GetComponent<GetahCollection>()
33                 .nextButton.transform.gameObject.SetActive(true);
34             collision.gameObject.GetComponent<DragController>().enabled = false;
35             stirrer.transform.gameObject.SetActive(true);
36         }
37     }
38 }
```

Full file (38 lines)

CheckContent.cs

Assets/CheckContent.cs · 64 lines · used in: Experiments

Checks (every frame) whether items in a list are active/inactive and then shows up to three objects.

- Known issue: the counter l is reset inside the loop and compared to i, so the condition is fragile; toAppear[1] and [2] throw if the list is shorter than 3.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class CheckContent : MonoBehaviour
{
    public List<Transform> item;

    public List<Transform> toAppear;

    public int l;

    public bool toDisappear;

    // Start is called before the first frame update
    void Start() { }

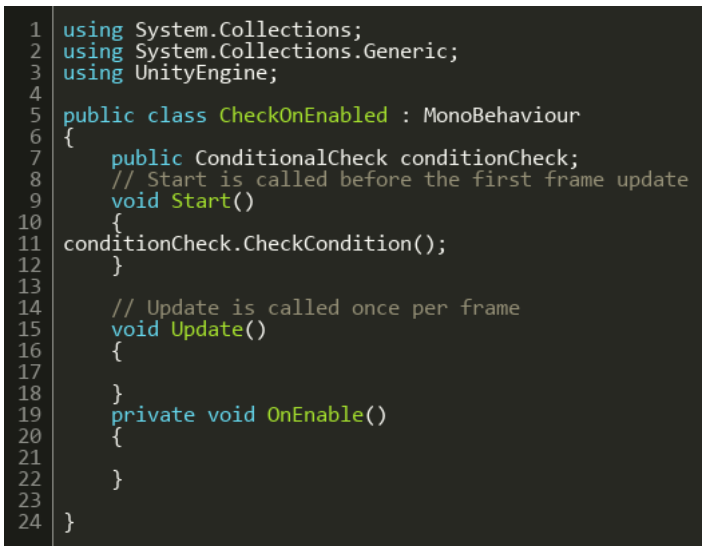
    // Update is called once per frame
    void Update()
    {
        for (int i = 0; i < item.Count; i++)
        {
            l = 0;
            if (!toDisappear)
            {
                if (!item[i].gameObject.activeSelf)
                {
                    l++;
                    if (i == l)
                    {
                        toAppear[0].gameObject.SetActive(true);
                        if (toAppear[1] != null)
                        {
                            toAppear[1].gameObject.SetActive(true);
                        }
                        if (toAppear[2] != null)
                        {
                            toAppear[2].gameObject.SetActive(true);
                        }
                    }
                }
            }
            else
            {
                if (item[i].gameObject.activeSelf)
                {
                    l++;
                    if (i == l)
                    {
                        toAppear[0].gameObject.SetActive(true);
                        if (toAppear[1] != null)
                        {
                            toAppear[1].gameObject.SetActive(true);
                        }
                        if (toAppear[2] != null)
                        {
                            toAppear[2].gameObject.SetActive(true);
                        }
                    }
                }
            }
        }
    }
}
```

Full file (64 lines)

CheckOnEnabled.cs

Assets/CheckOnEnabled.cs · 24 lines · used in: Experiments

On Start calls ConditionalCheck.CheckCondition() so a step is evaluated as soon as its panel appears.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class CheckOnEnabled : MonoBehaviour
{
    public ConditionalCheck conditionCheck;
    // Start is called before the first frame update
    void Start()
    {
conditionCheck.CheckCondition();
    }

    // Update is called once per frame
    void Update()
    {

    }
    private void OnEnable()
    {

    }

}
```

Full file (24 lines)

ConditionalCheck.cs

Assets/ConditionalCheck.cs · 84 lines · used in: All experiments (one per procedure panel)

Procedure-step checker. Decides when the 'next' button and the 'correct' marker are shown.

- procedurNum is parsed from the GameObject's name (so the panel must be named with a number).
- Condition enum: Number, Percent, Text, Transform. Only Percent (shows next button) and Transform (shows next button when every conditionToMeet is active) have behaviour; Number and Text are empty.
- CheckCondition() always ends by activating correctButton.
- Other scripts (triggerHit, DisplayItem, SuhuBilikControl) call CheckCondition() when their step is complete.



```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

public class ConditionalCheck : MonoBehaviour
{
    public int procedurNum;

    [Tooltip("Type of fading")]
    public Condi condition;

    private void Awake() { }

    [SerializeField]
    private List<Transform> conditionToMeet;

    [SerializeField]
    private Button nextButton;

    [SerializeField]
    private Transform correctButton;

    public enum Condi
    {
        Number,
        Percent,
        Text,

        Transform,
    }

    // Start is called before the first frame update
    void Start()
    {
        procedurNum = Int32.Parse(transform.name);
    }

    public void CheckCondition()
    {
        switch (condition)
        {
            //This statement works
            case Condi.Number: //Only this piece of code works
                break;

            //This statement doesn't work
            case Condi.Percent:

                //Debug.Log("continue");
                nextButton.GetComponent<Transform>().gameObject.SetActive(true);
                break;

            //This statement doesn't work
            case Condi.Text:
                break;

            //This statement doesn't work
            case Condi.Transform:
                bool isContinue = true;
                for (int i = 0; i < conditionToMeet.Count; i++)
                {
                    if (!conditionToMeet[i].gameObject.activeSelf)
                    {
                        Debug.Log("stop");
                        isContinue = false;
                        break;
                    }
                }

                if (isContinue)
                {
                    Debug.Log(isContinue);
                    nextButton.GetComponent<Transform>().gameObject.SetActive(true);
                }
                break;
        }
        correctButton.gameObject.SetActive(true);
    }

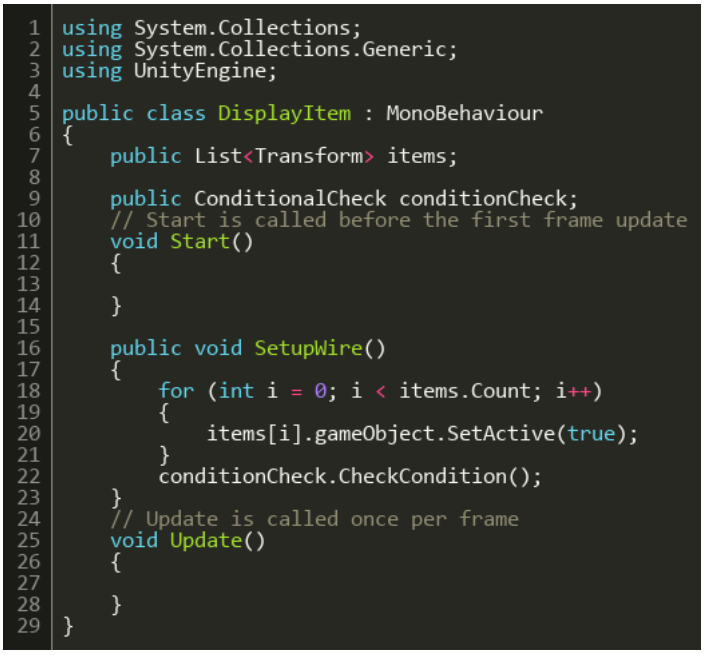
    // Update is called once per frame
    void Update() { }
}
```

Full file (84 lines)

DisplayItem.cs

Assets/DisplayItem.cs · 29 lines · used in: Experiments (wire/connection steps)

SetupWire() activates all items in a list and then asks ConditionalCheck to evaluate the step.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DisplayItem : MonoBehaviour
{
    public List<Transform> items;

    public ConditionalCheck conditionCheck;
    // Start is called before the first frame update
    void Start()
    {

    }

    public void SetupWire()
    {
        for (int i = 0; i < items.Count; i++)
        {
            items[i].gameObject.SetActive(true);
        }
        conditionCheck.CheckCondition();
    }
    // Update is called once per frame
    void Update()
    {

    }
}
```

Full file (29 lines)

ToDisplayContent.cs

Assets/ToDisplayContent.cs · 41 lines · used in: Experiments

Every frame, when all items in 'items' are inactive, activates all items in 'toDisplayItem'.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class ToDisplayContent : MonoBehaviour
{
    public List<Transform> items;

    public List<Transform> toDisplayItem;

    public bool allInactive;

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update()
    {
        for (int i = 0; i < items.Count; i++)
        {
            if (items[i].gameObject.activeSelf)
            {
                allInactive = false;
                break;
            }
            else
            {
                allInactive = true;
            }
        }

        if (allInactive)
        {
            for (int i = 0; i < toDisplayItem.Count; i++)
            {
                toDisplayItem[i].gameObject.SetActive(true);
            }
            Debug.Log("All objects are inactive!");
        }
    }
}
```

Full file (41 lines)

ObjectController.cs

Assets/ObjectController.cs · 25 lines · used in: Experiments (button events)

HideObject() deactivates its own GameObject; EnableObject() activates a referenced object. Meant for UnityEvents on buttons.

```
using System.Collections;
using System.Collections.Generic;
using Unity.VisualScripting;
using UnityEngine;

public class ObjectController : MonoBehaviour
{
    public Transform objeck;

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update() { }

    public void HideObject()
    {
        gameObject.SetActive(false);
    }

    public void EnableObject()
    {
        objeck.gameObject.SetActive(true);
    }
}
```

Full file (25 lines)

toggleController.cs

Assets/toggleController.cs · 39 lines · used in: Experiments (toggle buttons)

toggleOn() flips the visibility of object1 on each click.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class toggleController : MonoBehaviour
{
    private bool isToggle;

    public Transform object1;

    // Start is called before the first frame update
    void Start()
    {

    }

    public void toggleOn()
    {
        if (!isToggle)
        {
            object1.gameObject.SetActive(true);

            isToggle = true;
        }
        else
        {

            object1.gameObject.SetActive(false);

            isToggle = false;
        }
    }

    // Update is called once per frame
    void Update()
    {

    }
}
```

Full file (39 lines)

FallingObject.cs

Assets/FallingObject.cs · 48 lines · used in: Water simulation demo

Gives a Rigidbody2D a downward velocity and lets the arrow keys steer it; used to drop objects into the water.

- Uses Input.GetAxis (legacy input) and linearVelocity (Unity 6).
- Tag 'FallingObject' is checked by WaterSpring on collision.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class FallingObject : MonoBehaviour
{
    private Vector2 movementDirection;
    public Rigidbody2D rigidbody2D;

    [SerializeField]
    private float forceAmount;
    private float timeElapsed;
    private bool isMoving = false;

    void Start()
    {
        rigidbody2D.linearVelocity = Vector3.down * forceAmount;
    }

    void Update()
    {
        movementDirection = new Vector2(Input.GetAxis("Horizontal"), Input.GetAxis("Vertical"));
        timeElapsed += Time.deltaTime;

        if (
            Input.GetKeyDown(KeyCode.UpArrow)
            || Input.GetKeyDown(KeyCode.DownArrow)
            || Input.GetKeyDown(KeyCode.LeftArrow)
            || Input.GetKeyDown(KeyCode.RightArrow)
        )
        {
            isMoving = true;
        }
        if (
            Input.GetKeyUp(KeyCode.UpArrow)
            || Input.GetKeyUp(KeyCode.DownArrow)
            || Input.GetKeyUp(KeyCode.LeftArrow)
            || Input.GetKeyUp(KeyCode.RightArrow)
        )
        {
            isMoving = false;
        }
        if (isMoving)
        {
            rigidbody2D.linearVelocity = movementDirection * forceAmount;
        }
    }
}
```

Full file (48 lines)

3.4 Experiment-specific

Scripts that belong to a single experiment or to timing/temperature displays.

WaterVolumeController.cs

Assets/Scenes/Experiments/akt4/WaterVolumeController.cs · 28 lines · used in: akt4

AddLevel() activates the first inactive water-level object in the list, raising the visible liquid level one step.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class WaterVolumeController : MonoBehaviour
{
    public List<Transform> waterLevel;

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update() { }

    public void AddLevel()
    {
        for (int i = 0; i < waterLevel.Count; i++)
        {
            if (!waterLevel[i].gameObject.activeSelf)
            {
                waterLevel[i].gameObject.SetActive(true);
                break;
            }

            if (waterLevel[i].gameObject.activeSelf) { }
        }
    }
}
```

Full file (28 lines)

GetahCollection.cs

Assets/Scenes/Experiments/akt7/GetahCollection.cs · 57 lines · used in: akt7 (latex / getah collection)

Steps through a list of collection items one at a time.

- Start shows items[0]. clearAndNextItem() hides it, removes it from the list and shows the next.
- When one item is left the next button reads 'TUNGGU 2 JAM' ('wait 2 hours'); when none are left NextChapter() runs after 1 s, hiding fromPrev and showing ToNext.

```
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.UI;

public class GetahCollection : MonoBehaviour
{
    public static GetahCollection instance;

    public Transform nextButton;

    void Awake()
    {
        instance = this;
    }

    public Transform fromPrev;
    public Transform ToNext;

    public List<Transform> items;

    // Start is called before the first frame update
    void Start()
    {
        items[0].transform.gameObject.SetActive(true);
    }

    public void clearAndNextItem()
    {
        nextButton.gameObject.SetActive(false);
        items[0].transform.gameObject.SetActive(false);
        items.RemoveAt(0);
        if (items.Count > 0)
        {
            items[0].transform.gameObject.SetActive(true);
        }
        if (items.Count == 1)
        {
            nextButton.transform.GetChild(0).GetComponent<TextMeshProUGUI>().text = "TUNGGU 2 JAM";
        }

        if (items.Count == 0)
        {
            Invoke("NextChapter", 1f);
        }
    }

    public void NextChapter()
    {
        ToNext.gameObject.SetActive(true);
        fromPrev.gameObject.SetActive(false);
    }

    // Update is called once per frame
    void Update() { }
}
```

Full file (57 lines)

SuhuBilikControl.cs

Assets/SuhuBilikControl.cs · 123 lines · used in: akt3 (temperature control)

Drives the temperature slider while a flame ('api') heats an item during procedures 4 and 11.

- Each frame, if triggerHit.isCooldown is on: counts TimeTotal down and, on each tick, raises the slider by one and updates its label.
- At the slider's max value it stops the cooldown, locks the item and calls ConditionalCheck.CheckCondition().
- The procedure 4 and 11 branches are exact copies; the long parent chains (parent.parent.parent.GetChild(1)) are brittle and can be cached once.

```
15      void Update()
16      {
17          if (triggerHit.isCooldown)
18          {
19              if (conditionalCheck.procedurNum == 4)
20              {
21                  if (triggerHit.objectName.Equals("api"))
22                  {
23                      triggerHit.TimeTotal -= Time.deltaTime;
24                      if (triggerHit.TimeTotal < 0)
25                      {
26                          triggerHit
27                              .transform.parent.transform.parent.transform.parent.transform.GetChild(
28                                  1
29                              )
30                              .GetComponent<Slider>()
31                              .value = triggerHit.i;
32                          triggerHit.i++;
33                          //Debug.Log("apiTrigger");
34
35                          if (
36                              triggerHit.i
37                              <= triggerHit
38                                  .transform.parent.transform.parent.transform.parent.transform.GetChild(
39                                      1
40                                  )
41                                  .GetComponent<Slider>()
42                                  .maxValue
43                          )
44                          {
45                              triggerHit
46                                  .transform.parent.transform.parent.transform.parent.transform.GetChild(
47                                      1
48                                  )
49                                  .GetComponent<TextSlider>()
50                                  .text.text = triggerHit.i.ToString();
51                              if (
52                                  triggerHit.i
53                                  == triggerHit
54                                      .transform.parent.transform.parent.transform.parent.transform.GetChild(
55                                          1
56                                      )
```

Lines 15-56 of 123

TimerText.cs

Assets/TimerText.cs · 83 lines · used in: Experiments with a clock or thermometer

Accelerated clock or warming thermometer shown in a TextMeshPro label.

- Timer mode (isTimer): elapsed time runs timeMultiplier (22x) faster and shows M:SS; stops at 1:00 (isMinute), 0:45 (is45min) or 0:30.
- Temperature mode: currentTemp rises at 'speed' per second until targetTemp (default 35 to 37 C) and shows e.g. '36°C'.



```
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;

public class TimerText : MonoBehaviour
{
    public TextMeshProUGUI timerText; // Assign a UI Text (or TMP_Text) in Inspector
    public float elapsedTime = 0f;
    public float timeMultiplier = 22f;
    private bool counting = true;

    public bool isMinute;
    public bool is45min;

    public bool isTimer;

    [Space]
    public float currentTemp = 35f;
    public float speed = 5f; // how fast it rises (adjust as needed)
    public float targetTemp = 37f;

    void Update()
    {
        if (counting)
        {
            elapsedTime += Time.deltaTime * timeMultiplier;

            if (isTimer)
            {
                // Convert seconds into minutes and seconds
                int minutes = Mathf.FloorToInt(elapsedTime / 60f);
                int seconds = Mathf.FloorToInt(elapsedTime % 60f);

                // Format MM:SS
                timerText.text = string.Format("{0:0}:{1:00}", minutes, seconds);

                if (isMinute)
                {
                    // Stop at 1:00 (one hour)
                    if (minutes >= 1 && seconds >= 0)
                    {
                        counting = false;
                        timerText.text = "1:00"; // Lock display at 1:00
                    }
                }
                else if (is45min)
                {
                    // Stop at 0:45 (45 minutes)
                    if (minutes == 0 && seconds >= 45)
                    {
                        counting = false;
                        timerText.text = "0:45"; // Lock display at 1:00
                    }
                }
                else
                {
                    // Stop at 0:30 (30 minutes)
                    if (minutes == 0 && seconds >= 30)
                    {
                        counting = false;
                        timerText.text = "0:30"; // Lock display at 1:00
                    }
                }
            }
            else
            {
                if (currentTemp < targetTemp)
                {
                    // Increase temperature smoothly
                    currentTemp += Time.deltaTime * speed;

                    // Clamp so it doesn't overshoot
                    if (currentTemp > targetTemp)
                        currentTemp = targetTemp;

                    // Display as integer (no decimals)
                    timerText.text = Mathf.FloorToInt(currentTemp).ToString() + "°C";
                }
            }
        }
    }
}
```

Full file (83 lines)

3.5 Visual and Audio

Water simulation, music and small UI helpers.

WaterShapeController.cs

Assets/WaterShapeController.cs · 191 lines · used in: Water scenes (SpriteShape)

Generates and animates a wavy water surface on a SpriteShapeController ([ExecuteAlways]).

- WavesCount spring points are created (OnValidate -> CreateWaves -> SetWaves) and added to the spline, with Smoothen() rounding the tangents.
- FixedUpdate: each WaterSpring runs WaveSpringUpdate/WavePointUpdate, then UpdateSprings() spreads velocity to neighbours (spread).
- Splash(index, speed) adds velocity to one spring.
- Tunables: springStiffness 0.1, dampening 0.03, spread 0.006.

```
152     void FixedUpdate()
153     {
154         foreach (WaterSpring waterSpringComponent in springs)
155         {
156             waterSpringComponent.WaveSpringUpdate(springStiffness, dampening);
157             waterSpringComponent.WavePointUpdate();
158         }
159 
160         UpdateSprings();
161     }
162 
163     private void UpdateSprings()
164     {
165         int count = springs.Count;
166         float[] left_deltas = new float[count];
167         float[] right_deltas = new float[count];
168 
169         for (int i = 0; i < count; i++)
170         {
171             if (i > 0)
172             {
173                 left_deltas[i] = spread * (springs[i].height - springs[i - 1].height);
174                 springs[i - 1].velocity += left_deltas[i];
175             }
176             if (i < springs.Count - 1)
177             {
178                 right_deltas[i] = spread * (springs[i].height - springs[i + 1].height);
179                 springs[i + 1].velocity += right_deltas[i];
180             }
181         }
182     }
183 
184     private void Splash(int index, float speed)
185     {
186         if (index >= 0 && index < springs.Count)
187         {
188             springs[index].velocity += speed;
189         }
190     }
191 }
```

Lines 152-191 of 191

WaterSpring.cs

Assets/WaterSpring.cs · 78 lines · used in: Water scenes

One point on the water surface, simulated as a damped spring (Hooke's law with dampening).

- Init links the point to the spline index (sibling index + 1).
- WaveSpringUpdate: force = -stiffness * displacement - dampening * velocity; updates velocity and height.
- WavePointUpdate writes the new height into the spline.
- OnCollisionEnter2D: an object tagged 'FallingObject' adds velocity (vy / 40) so splashes appear.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.U2D;

public class WaterSpring : MonoBehaviour
{
    public float velocity = 0;
    public float force = 0;

    // current height
    public float height = 0f;

    // normal height
    private float target_height = 0f;
    public Transform springTransform;

    [SerializeField]
    private SpriteShapeController spriteShapeController = null;
    private int waveIndex = 0;
    private List<WaterSpring> springs = new();
    private float resistance = 40f;

    public void Init(SpriteShapeController ssc)
    {
        var index = transform.GetSiblingIndex();
        waveIndex = index + 1;

        spriteShapeController = ssc;
        velocity = 0;
        height = transform.localPosition.y;
        target_height = transform.localPosition.y;
    }

    // with dampening
    // adding the dampening to the force
    public void WaveSpringUpdate(float springStiffness, float dampening)
    {
        height = transform.localPosition.y;
        // maximum extension
        var x = height - target_height;
        var loss = -dampening * velocity;

        force = -springStiffness * x + loss;
        velocity += force;
        var y = transform.localPosition.y;
        transform.localPosition = new Vector3(
            transform.localPosition.x,
            y + velocity,
            transform.localPosition.z
        );
    }

    public void WavePointUpdate()
    {
        if (spriteShapeController != null)
        {
            Spline waterSpline = spriteShapeController.spline;
            Vector3 wavePosition = waterSpline.GetPosition(waveIndex);
            waterSpline.SetPosition(
                waveIndex,
                new Vector3(wavePosition.x, transform.localPosition.y, wavePosition.z)
            );
        }
    }

    private void OnCollisionEnter2D(Collision2D other)
    {
        if (other.gameObject.tag.Equals("FallingObject"))
        {
            FallingObject fallingObject = other.gameObject.GetComponent<FallingObject>();
            Rigidbody2D rb = fallingObject.GetComponent<Rigidbody2D>();
            var speed = rb.linearVelocity;

            velocity += speed.y / resistance;
        }
    }
}
```

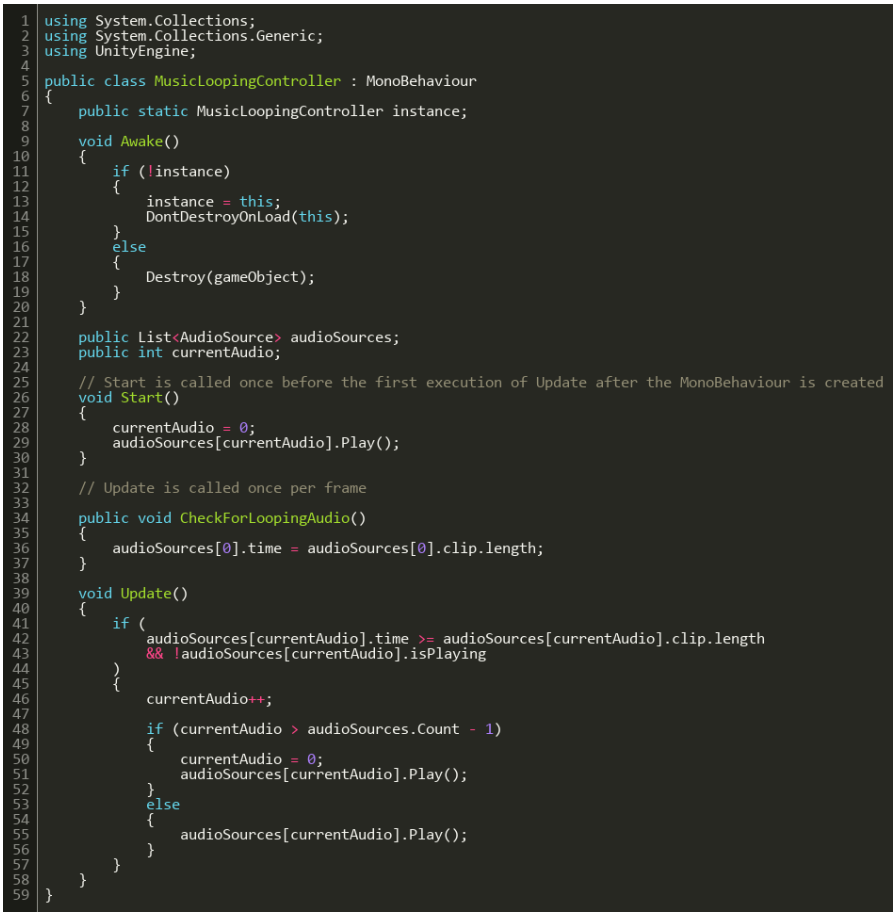
Full file (78 lines)

MusicLoopingController.cs

Assets/MusicLoopingController.cs · 59 lines · used in: All scenes (persistent audio)

Persistent singleton that plays a playlist of AudioSources one after another and loops back to the first.

- Update checks whether the current track has finished and starts the next.
- CheckForLoopingAudio() jumps audioSources[0] to its end so the playlist advances.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class MusicLoopingController : MonoBehaviour
{
    public static MusicLoopingController instance;

    void Awake()
    {
        if (!instance)
        {
            instance = this;
            DontDestroyOnLoad(this);
        }
        else
        {
            Destroy(gameObject);
        }
    }

    public List<AudioSource> audioSources;
    public int currentAudio;

    // Start is called once before the first execution of Update after the MonoBehaviour is created
    void Start()
    {
        currentAudio = 0;
        audioSources[currentAudio].Play();
    }

    // Update is called once per frame

    public void CheckForLoopingAudio()
    {
        audioSources[0].time = audioSources[0].clip.length;
    }

    void Update()
    {
        if (
            audioSources[currentAudio].time >= audioSources[currentAudio].clip.length
            && !audioSources[currentAudio].isPlaying
        )
        {
            currentAudio++;

            if (currentAudio > audioSources.Count - 1)
            {
                currentAudio = 0;
                audioSources[currentAudio].Play();
            }
            else
            {
                audioSources[currentAudio].Play();
            }
        }
    }
}
```

Full file (59 lines)

TextSlider.cs

Assets/TextSlider.cs · 20 lines · used in: Sliders with a value label

Finds the child named 'text' and exposes it as a TextMeshProUGUI so other scripts can show the slider value.

```
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;

public class TextSlider : MonoBehaviour
{
    public TextMeshProUGUI text;
    // Start is called before the first frame update
    void Start()
    {
        text = transform.Find("text").GetComponent<TextMeshProUGUI>();
    }

    // Update is called once per frame
    void Update()
    {

    }
}
```

Full file (20 lines)

transformObject.cs

Assets/Animation/transformObject.cs · 17 lines · used in: Animation events

DisableObject() deactivates its GameObject; called from animation events when a clip ends.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class transformObject : MonoBehaviour
{
    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update() { }

    public void DisableObject()
    {
        transform.gameObject.SetActive(false);
    }
}
```

Full file (17 lines)

4. Appendix - Full Source of the Largest Scripts

triggerHit.cs

```
using System.Collections;
using System.Collections.Generic;
using TMPro;
using Unity.VisualScripting;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

public class triggerHit : MonoBehaviour
{
    public ConditionalCheck conditionalCheck;

    [SerializeField]
    public string objectName;

    [SerializeField]
    public float i = 0.1f;

    public float TimeTotal;
    public float tempTime;

    public bool isCooldown;

    // Start is called before the first frame update
    void Start()
    {
        tempTime = TimeTotal;
        objectName = GetComponent<Transform>().transform.name;
    }

    // Update is called once per frame
    void Update() { }

    public void Procedur13()
    {
        GameObject.Find("TempSlider").GetComponent<Slider>().value = i;
        i += 10;
        //Debug.Log("apiTrigger");

        if (i <= GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
        {
            if (i > 40 && i < 60)
            {
                GameObject
                    .Find("Petri")
                    .GetComponent<Transform>()
                    .GetChild(0)
                    .GetComponent<CanvasGroup>()
                    .alpha = 0.5f;
            }
            else if (i > 60)
            {
                GameObject
                    .Find("Petri")
                    .GetComponent<Transform>()
                    .GetChild(0)
                    .GetComponent<CanvasGroup>()
                    .alpha = 1f;
            }
            GameObject.Find("TempSlider").GetComponent<TextSlider>().text.text = i.ToString();
            if (i == GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
            {
                isCooldown = false;

                conditionalCheck.CheckCondition();
            }
        }
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (SceneManager.GetActiveScene().name.Equals("akt3"))
        {
            if (conditionalCheck.procedurNum == 2)
            {
                if (objectName.Equals("api"))
                {
                    transform
                        .parent.transform.parent.transform.GetChild(1)
                        .gameObject.SetActive(true);
                    //Debug.Log("onEnter");
                }
            }

            if (conditionalCheck.procedurNum == 5)
            {
                if (objectName.Equals("larutanKultur"))
                {
```

```
                    transform
                        .parent.transform.parent.transform.GetChild(1)
                        .gameObject.SetActive(true);
                    //Debug.Log("onEnter");
                }
            }
            if (conditionalCheck.procedurNum == 6)
            {
                if (objectName.Equals("collider"))
                {
                    GameObject.Find("TempSlider").GetComponent<Slider>().value = i;
                    i += 10;

                    if (i > 60)
                        transform.parent.GetComponent<CanvasGroup>().alpha = 1f;

                    if (i >= GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                    {
                        i = 100;
                        conditionalCheck.CheckCondition();
                    }
                    GameObject.Find("TempSlider").GetComponent<TextSlider>().text.text = i.ToString(
                        "00"
                    );
                    Debug.Log("zigzag tuch");
                }
            }
            if (conditionalCheck.procedurNum == 7)
            {
                if (objectName.Equals("collider"))
                {
                    GameObject.Find("TempSlider").GetComponent<Slider>().value = i;
                    i += 10;

                    if (i > 60)
                        transform.parent.GetComponent<CanvasGroup>().alpha = 1f;

                    if (i >= GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                    {
                        i = 100;
                        conditionalCheck.CheckCondition();
                    }
                    GameObject.Find("TempSlider").GetComponent<TextSlider>().text.text = i.ToString(
                        "00"
                    );
                    Debug.Log("zigzag tuch");
                }
            }
            if (conditionalCheck.procedurNum == 8)
            {
                if (objectName.Equals("collider"))
                {
                    GameObject.Find("TempSlider").GetComponent<Slider>().value = i;
                    i += 10;

                    if (i > 60)
                        transform.parent.GetComponent<CanvasGroup>().alpha = 1f;

                    if (i >= GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                    {
                        i = 100;
                        conditionalCheck.CheckCondition();
                    }
                    GameObject.Find("TempSlider").GetComponent<TextSlider>().text.text = i.ToString(
                        "00"
                    );
                    Debug.Log("zigzag tuch");
                }
            }

            if (conditionalCheck.procedurNum == 9)
            {
                if (objectName.Equals("collider"))
                {
                    GameObject.Find("TempSlider").GetComponent<Slider>().value = i;
                    i += 10;

                    if (i > 60)
                        transform.parent.GetComponent<CanvasGroup>().alpha = 1f;

                    if (i >= GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                    {
                        i = 100;
                        conditionalCheck.CheckCondition();
                    }
                    GameObject.Find("TempSlider").GetComponent<TextSlider>().text.text = i.ToString(
                        "00"
                    );
                    Debug.Log("zigzag tuch");
                }
            }
        }
        else if (SceneManager.GetActiveScene().name.Equals("akt7"))
        {
            if (conditionalCheck.procedurNum == 1)
```

```
            {
                if (other.gameObject.tag == "action")
                {
                    Debug.Log("actionproduct");
                }
            }
        }
    }

    private void OnTriggerStay2D(Collider2D other)
    {
        if (SceneManager.GetActiveScene().name.Equals("akt3"))
        {
            if (conditionalCheck.procedurNum == 3)
            {
                if (objectName.Equals("api"))
                {
                    TimeTotal -= Time.deltaTime;
                    if (TimeTotal < 0)
                    {
                        transform
                            .parent.transform.parent.transform.parent.transform.GetChild(1)
                            .GetComponent<Slider>()
                            .value = i;
                        i++;
                        //Debug.Log("apiTrigger");

                        if (
                            i
                            <= transform
                                .parent.transform.parent.transform.parent.transform.GetChild(1)
                                .GetComponent<Slider>()
                                .maxValue
                        )
                        {
                            transform
                                .parent.transform.parent.transform.parent.transform.GetChild(1)
                                .GetComponent<TextSlider>()
                                .text.text = i.ToString();
                            if (
                                i
                                == transform
                                    .parent.transform.parent.transform.parent.transform.GetChild(1)
                                    .GetComponent<Slider>()
                                    .maxValue
                            )
                            {
                                isCooldown = false;

                                conditionalCheck.CheckCondition();
                            }
                        }

                        TimeTotal = tempTime;
                    }
                }
            }

            if (conditionalCheck.procedurNum == 5)
            {
                if (objectName.Equals("larutanKultur"))
                {
                    TimeTotal -= Time.deltaTime;
                    if (TimeTotal < 0)
                    {
                        GameObject.Find("TempSlider").GetComponent<Slider>().value = i;
                        i++;
                        //Debug.Log("apiTrigger");

                        if (i <= GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                        {
                            GameObject.Find("TempSlider").GetComponent<TextSlider>().text.text =
                                i.ToString();
                            if (i == GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                            {
                                isCooldown = false;

                                conditionalCheck.CheckCondition();
                            }
                        }

                        TimeTotal = tempTime;
                    }
                }
            }

            if (conditionalCheck.procedurNum == 10)
            {
                if (objectName.Equals("api"))
                {
                    TimeTotal -= Time.deltaTime;
                    if (TimeTotal < 0)
                    {
                        GameObject.Find("SterilSlider").GetComponent<Slider>().value = i;
                        i++;
```

```
                        //Debug.Log("apiTrigger");

                        if (i <= GameObject.Find("SterilSlider").GetComponent<Slider>().maxValue)
                        {
                            GameObject.Find("SterilSlider").GetComponent<TextSlider>().text.text =
                                i.ToString();
                            if (
                                i == GameObject.Find("SterilSlider").GetComponent<Slider>().maxValue
                            )
                            {
                                isCooldown = false;

                                conditionalCheck.CheckCondition();
                            }
                        }

                        TimeTotal = tempTime;
                    }
                }
            }
        }

        if (conditionalCheck.procedurNum == 12)
        {
            if (objectName.Equals("larutanKultur"))
            {
                TimeTotal -= Time.deltaTime;
                if (TimeTotal < 0)
                {
                    GameObject.Find("TempSlider").GetComponent<Slider>().value = i;
                    i++;
                    //Debug.Log("apiTrigger");

                    if (i <= GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                    {
                        GameObject.Find("TempSlider").GetComponent<TextSlider>().text.text =
                            i.ToString();
                        if (i == GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                        {
                            isCooldown = false;

                            conditionalCheck.CheckCondition();
                        }
                    }

                    TimeTotal = tempTime;
                }
            }
        }
        if (conditionalCheck.procedurNum == 14)
        {
            if (objectName.Equals("circle"))
            {
                TimeTotal -= Time.deltaTime;
                if (TimeTotal < 0)
                {
                    GameObject.Find("TempSlider").GetComponent<Slider>().value = i;
                    i += 5;
                    //Debug.Log("apiTrigger");

                    if (i <= GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                    {
                        GameObject.Find("TempSlider").GetComponent<TextSlider>().text.text =
                            i.ToString();
                        if (i == GameObject.Find("TempSlider").GetComponent<Slider>().maxValue)
                        {
                            isCooldown = false;

                            conditionalCheck.CheckCondition();
                        }
                    }

                    TimeTotal = tempTime;
                }
            }
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
        if (SceneManager.GetActiveScene().name.Equals("akt3"))
        {
            if (conditionalCheck.procedurNum == 4)
            {
                if (objectName.Equals("api"))
                {
                    //rujuk SuhuBilikControl.cs
                    isCooldown = true;
                }
            }
            if (conditionalCheck.procedurNum == 5)
            {
                if (objectName.Equals("larutanKultur"))
                {
                    //rujuk SuhuBilikControl.cs
```

```
                    isCooldown = true;
                }
            }

            if (conditionalCheck.procedurNum == 11)
            {
                if (objectName.Equals("api"))
                {
                    //rujuk SuhuBilikControl.cs
                    isCooldown = true;
                }
            }
        }
    }
}
```

ItemSlot.cs

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

public class ItemSlot : MonoBehaviour, IDropHandler
{
    public Transform tick;

    [Header("Akt6")]
    public string jenisAnodKatod;
    public Transform ConditionalCheck;

    public string nameScene;

    public Transform ButtonNext;

    // Start is called before the first frame update
    void Start()
    {
        tick = transform.GetChild(0).GetComponent<Transform>();
    }

    // Update is called once `r frame
    void Update() { }

    public void OnDrop(PointerEventData eventData)
    {
        nameScene = SceneManager.GetActiveScene().name.ToString();
        Debug.Log("onDrop");
        if (!SceneManager.GetActiveScene().name.Equals("akt6"))
        {
            //Debug.Log("stuck");
            eventData.pointerDrag.transform.SetParent(transform);
            //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition = GetComponent<RectTransform>().anchoredPosition;
            eventData.pointerDrag.transform.localPosition = new Vector3(0, 0, 0);

            tick.gameObject.SetActive(true);

            eventData.pointerDrag.transform.GetComponent<DragController>().enabled = false;

            if (SceneManager.GetActiveScene().name.Equals("akt5"))
            {
                eventData.pointerDrag.transform.localPosition = new Vector3(0, 100f, 0);
            }
        }
        else
        {
            Debug.Log("stuck");
        }
        GetComponent<Image>().enabled = false;
    }

    void OnTriggerStay2D(Collider2D collision)
    {
        if (SceneManager.GetActiveScene().name == "akt7")
        {
            if (ConditionalCheck.name == "3")
            {
                if (jenisAnodKatod == "anod")
                {
                    Debug.Log("anod");
                    if (collision.name.Equals("rubberfilm"))
                    {
                        collision.transform.GetComponent<DragController>().enabled = false;
                        //Debug.Log("stuck");
                        collision.transform.SetParent(transform);
                        //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
        GetComponent<RectTransform>().anchoredPosition;
                        collision.transform.localPosition = new Vector3(0, 0, 0);

                        tick.gameObject.SetActive(true);
                        ButtonNext.gameObject.SetActive(true);
```

```
                        //transform.gameObject.SetActive(false);
                    }
                }
            }
            if (ConditionalCheck.name == "2")
            {
                if (jenisAnodKatod == "anod")
                {
                    Debug.Log("anod");
                    if (collision.name.Equals("rubberfilm"))
                    {
                        collision.transform.GetComponent<DragController>().enabled = false;
                        //Debug.Log("stuck");
                        collision.transform.SetParent(transform);
                        //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
GetComponent<RectTransform>().anchoredPosition;
                        collision.transform.localPosition = new Vector3(0, 0, 0);

                        tick.gameObject.SetActive(true);
                        ButtonNext.gameObject.SetActive(true);
                        //transform.gameObject.SetActive(false);
                    }
                }
            }
        }
        if (ConditionalCheck.name == "0 (2)")
        {
            if (jenisAnodKatod == "anod")
            {
                Debug.Log("anod");
                if (collision.name.Equals("anodElektrod"))
                {
                    collision.transform.GetComponent<DragController>().enabled = false;
                    //Debug.Log("stuck");
                    collision.transform.SetParent(transform);
                    //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
GetComponent<RectTransform>().anchoredPosition;
                    collision.transform.localPosition = new Vector3(0, 0, 0);

                    tick.gameObject.SetActive(true);

                    //transform.gameObject.SetActive(false);
                }
            }
            else if (jenisAnodKatod == "anod2")
            {
                Debug.Log("anod");
                if (collision.name.Equals("anodElektrod2"))
                {
                    collision.transform.GetComponent<DragController>().enabled = false;
                    //Debug.Log("stuck");
                    collision.transform.SetParent(transform);
                    //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
GetComponent<RectTransform>().anchoredPosition;
                    collision.transform.localPosition = new Vector3(0, 0, 0);

                    tick.gameObject.SetActive(true);

                    //transform.gameObject.SetActive(false);
                }
            }
            else if (jenisAnodKatod == "katod")
            {
                Debug.Log("katod");
                if (collision.name.Equals("katodElektrod"))
                {
                    collision.transform.GetComponent<DragController>().enabled = false;
                    //Debug.Log("stuck");
                    collision.transform.SetParent(transform);
                    //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
GetComponent<RectTransform>().anchoredPosition;
                    collision.transform.localPosition = new Vector3(0, 0, 0);

                    tick.gameObject.SetActive(true);

                    //transform.gameObject.SetActive(false);
                }
            }
            else if (jenisAnodKatod == "katod2")
            {
                Debug.Log("katod2");
                if (collision.name.Equals("katodElektrod2"))
                {
                    collision.transform.GetComponent<DragController>().enabled = false;
                    //Debug.Log("stuck");
                    collision.transform.SetParent(transform);
                    //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
GetComponent<RectTransform>().anchoredPosition;
                    collision.transform.localPosition = new Vector3(0, 0, 0);

                    tick.gameObject.SetActive(true);

                    //transform.gameObject.SetActive(false);
                }
            }
```

```
165         }
166         else if (ConditionalCheck.name == "0 (3)")
167         {
168             if (jenisAnodKatod == "anod")
169             {
170                 Debug.Log("anod");
171                 if (collision.name.Equals("anodElektrod"))
172                 {
173                     collision.transform.GetComponent<DragController>().enabled = false;
174                     //Debug.Log("stuck");
175                     collision.transform.SetParent(transform);
176                     //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
        GetComponent<RectTransform>().anchoredPosition;
177                     collision.transform.localPosition = new Vector3(0, 0, 0);
178 
179                     tick.gameObject.SetActive(true);
180 
181                     //transform.gameObject.SetActive(false);
182                 }
183             }
184             else if (jenisAnodKatod == "katod")
185             {
186                 Debug.Log("katod");
187                 if (collision.name.Equals("katodElektrod"))
188                 {
189                     collision.transform.GetComponent<DragController>().enabled = false;
190                     //Debug.Log("stuck");
191                     collision.transform.SetParent(transform);
192                     //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
        GetComponent<RectTransform>().anchoredPosition;
193                     collision.transform.localPosition = new Vector3(0, 0, 0);
194 
195                     tick.gameObject.SetActive(true);
196 
197                     //transform.gameObject.SetActive(false);
198                 }
199             }
200         }
201         else if (ConditionalCheck.name == "0 (4)")
202         {
203             if (jenisAnodKatod == "anod")
204             {
205                 Debug.Log("anod");
206                 if (collision.name.Equals("anodElektrod"))
207                 {
208                     collision.transform.GetComponent<DragController>().enabled = false;
209                     //Debug.Log("stuck");
210                     collision.transform.SetParent(transform);
211                     //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
        GetComponent<RectTransform>().anchoredPosition;
212                     collision.transform.localPosition = new Vector3(0, 0, 0);
213 
214                     tick.gameObject.SetActive(true);
215 
216                     //transform.gameObject.SetActive(false);
217                 }
218             }
219             else if (jenisAnodKatod == "katod")
220             {
221                 Debug.Log("katod");
222                 if (collision.name.Equals("katodElektrod"))
223                 {
224                     collision.transform.GetComponent<DragController>().enabled = false;
225                     //Debug.Log("stuck");
226                     collision.transform.SetParent(transform);
227                     //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
        GetComponent<RectTransform>().anchoredPosition;
228                     collision.transform.localPosition = new Vector3(0, 0, 0);
229 
230                     tick.gameObject.SetActive(true);
231 
232                     //transform.gameObject.SetActive(false);
233                 }
234             }
235         }
236         else if (ConditionalCheck.name == "0 (8)")
237         {
238             if (jenisAnodKatod == "anod")
239             {
240                 Debug.Log("anod");
241                 if (collision.name.Equals("anodElektrod"))
242                 {
243                     collision.transform.GetComponent<DragController>().enabled = false;
244                     //Debug.Log("stuck");
245                     collision.transform.SetParent(transform);
246                     //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
        GetComponent<RectTransform>().anchoredPosition;
247                     collision.transform.localPosition = new Vector3(0, 0, 0);
248 
249                     tick.gameObject.SetActive(true);
250                     ButtonNext.gameObject.SetActive(true);
251                     //transform.gameObject.SetActive(false);
252                 }
253             }
254             else if (jenisAnodKatod == "katod")
```

```
            {
                Debug.Log("katod");
                if (collision.name.Equals("katodElektrod"))
                {
                    collision.transform.GetComponent<DragController>().enabled = false;
                    //Debug.Log("stuck");
                    collision.transform.SetParent(transform);
                    //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
        GetComponent<RectTransform>().anchoredPosition;
                    collision.transform.localPosition = new Vector3(0, 0, 0);

                    tick.gameObject.SetActive(true);
                    ButtonNext.gameObject.SetActive(true);
                    //transform.gameObject.SetActive(false);
                }
            }
        }
        else if (ConditionalCheck.name == "2")
        {
            if (jenisAnodKatod == "anod")
            {
                if (collision.name.Equals("wayarAnod"))
                {
                    Debug.Log("anod2");
                    collision.transform.GetComponent<DragController>().enabled = false;
                    //Debug.Log("stuck");
                    collision.transform.SetParent(transform);
                    //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
        GetComponent<RectTransform>().anchoredPosition;
                    collision.transform.localPosition = new Vector3(0, 0, 0);

                    tick.gameObject.SetActive(true);

                    //transform.gameObject.SetActive(false);
                }
            }
            else if (jenisAnodKatod == "katod")
            {
                if (collision.name.Equals("watarKatod"))
                {
                    Debug.Log("katod2");
                    collision.transform.GetComponent<DragController>().enabled = false;
                    //Debug.Log("stuck");
                    collision.transform.SetParent(transform);
                    //eventData.pointerDrag.GetComponent<RectTransform>().anchoredPosition =
        GetComponent<RectTransform>().anchoredPosition;
                    collision.transform.localPosition = new Vector3(0, 0, 0);

                    tick.gameObject.SetActive(true);

                    //transform.gameObject.SetActive(false);
                }
            }
            ConditionalCheck.GetComponent<ConditionalCheck>().CheckCondition();
        }
    }
}
```

QuestionController.cs

```
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.SocialPlatforms.Impl;
using UnityEngine.UI;

public class QuestionController : MonoBehaviour
{
    void Awake() { }

    public TextMeshProUGUI questionNumberTopLabel;
    public TextMeshProUGUI questionLabel;

    public TextMeshProUGUI langkahLabel;
    public List<Transform> answerBtn;
    public Transform avatarState;

    public Transform CongratPrompt;
    public int countQuestion;
    public int countStep;

    void defaultAvatarFace()
    {
        avatarState.GetChild(0).gameObject.SetActive(true);
        avatarState.GetChild(1).gameObject.SetActive(false);
        avatarState.GetChild(2).gameObject.SetActive(false);
    }

    void AvatarSmile()
    {
        avatarState.GetChild(0).gameObject.SetActive(false);
        avatarState.GetChild(1).gameObject.SetActive(true);
```

```
        avatarState.GetChild(2).gameObject.SetActive(false);
    }

    void AvatarSad()
    {
        avatarState.GetChild(0).gameObject.SetActive(false);
        avatarState.GetChild(1).gameObject.SetActive(false);
        avatarState.GetChild(2).gameObject.SetActive(true);
    }

    // Start is called before the first frame update
    void Start()
    {
        countQuestion = 0;
        countStep = 0;

        Debug.Log(QuestionList.instance.questionlist[countQuestion].question[countStep]);
        applyQuestion();
        applyAnswerOnButton();
    }

    private void applyQuestion()
    {
        questionNumberTopLabel.text = "Soalan " + (countQuestion + 1);
        questionLabel.text = QuestionList.instance.questionlist[countQuestion].question[0];
    }

    private void applyAnswerOnButton()
    { //disable all buttons first
        /*
            switch (countStep)
            {
                case 0:
                    langkahLabel.text = "Langkah 1 : Maklumat diberi";
                    break;
                case 1:
                    langkahLabel.text = "Langkah 2 : Operasi";
                    break;
                case 2:
                    langkahLabel.text = "Langkah 3 : Selesaikan ";
                    break;
                case 3:
                    langkahLabel.text = "Langkah 4 : Semak";
                    break;
                case 4:
                    langkahLabel.text = "Jawapan : ";
                    break;
            }
          */for (int i = 0; i < answerBtn.Count; i++)
        {
            answerBtn[i].gameObject.SetActive(false);
        }
        //check for answes count
        for (
            int i = 0;
            i
                < QuestionList
                    .instance
                    .questionlist[countQuestion]
                    .QuestionOnAnswersBtn[countStep]
                    .answer
                    .Count;
            i++
        )
        {
            answerBtn[i].gameObject.SetActive(true);
        }
        //assign text answer on all buttons
        for (
            int i = 0;
            i
                < QuestionList
                    .instance
                    .questionlist[countQuestion]
                    .QuestionOnAnswersBtn[countStep]
                    .answer
                    .Count;
            i++
        )
        {
            answerBtn[i].GetComponent<Button>().onClick.RemoveAllListeners();

            answerBtn[i].GetChild(0).GetComponent<TextMeshProUGUI>().text = QuestionList
                .instance
                .questionlist[countQuestion]
                .QuestionOnAnswersBtn[countStep]
                .answer[i];

            if (
                QuestionList
                    .instance
                    .questionlist[countQuestion]
                    .QuestionOnAnswersBtn[countStep]
                    .answeOnBtn == i
            )
```

```
            {
                Debug.Log(
                    QuestionList
                        .instance
                        .questionlist[countQuestion]
                        .QuestionOnAnswersBtn[countStep]
                        .answeOnBtn
                );
                answerBtn[i].GetComponent<Button>().onClick.AddListener(() => AddMark());
                answerBtn[i].GetComponent<Button>().onClick.AddListener(() => NextStepping());
            }
            else
            {
                answerBtn[i].GetComponent<Button>().onClick.AddListener(() => IncorrectButton());
                answerBtn[i].GetComponent<Button>().onClick.AddListener(() => NextStepping());
            }
        }
    }

    void disableAllButtons(bool boolean)
    {
        //turn everybutton to disable
        for (int i = 0; i < answerBtn.Count; i++)
        {
            answerBtn[i].GetComponent<Button>().enabled = boolean;
        }
    }

    void showRealAnswer()
    {
        for (int i = 0; i < answerBtn.Count; i++)
        {
            if (
                QuestionList
                    .instance
                    .questionlist[countQuestion]
                    .QuestionOnAnswersBtn[countStep]
                    .answeOnBtn == i
            )
            {
                answerBtn[i].GetComponent<Image>().color = new Color32(145, 187, 21, 255);
            }
            else
            {
                answerBtn[i].GetComponent<Image>().color = Color.red;
            }
        }
    }

    void resetButton()
    {
        for (int i = 0; i < answerBtn.Count; i++)
        {
            answerBtn[i].GetComponent<Image>().color = new Color32(145, 187, 21, 255);
        }
    }

    public void AddMark()
    {
        //turn everybutton to disable
        disableAllButtons(false);

        StartCoroutine(addingmark());
    }

    public void IncorrectButton()
    { //turn everybutton to disable
        disableAllButtons(false);

        StartCoroutine(incorrect());
    }

    IEnumerator incorrect()
    {
        yield return new WaitForSeconds(1);
        if (!GameObject.Find("salah").GetComponent<AudioSource>().isPlaying)
        {
            GameObject.Find("salah").GetComponent<AudioSource>().Play();
        }
        //sad
        AvatarSad();
    }

    IEnumerator addingmark()
    {
        yield return new WaitForSeconds(1);
        if (!GameObject.Find("yay").GetComponent<AudioSource>().isPlaying)
        {
            GameObject.Find("yay").GetComponent<AudioSource>().Play();
        }
        if (!GameObject.Find("claps").GetComponent<AudioSource>().isPlaying)
        {
            GameObject.Find("claps").GetComponent<AudioSource>().Play();
        }

```

```
        //smile
        AvatarSmile();

        //play animation add coin and change text
        //        GameObject.Find("Coin").GetComponent<Animator>().Play("addCoin");
        yield return new WaitForSeconds(1.3f);
        ScoreController.instance.score += 10;
        ScoreController.instance.SetScore();
    }

    IEnumerator nextStep()
    {
        //turn everybutton to red
        for (int i = 0; i < answerBtn.Count; i++)
        {
            answerBtn[i].GetComponent<Button>().enabled = false;
        }
        yield return new WaitForSeconds(1.2f);
        showRealAnswer();
        countStep++;
        yield return new WaitForSeconds(3);
        countQuestion++;
        if (countQuestion >= QuestionList.instance.questionlist.Count)
        {
            //countStep++;
            // applyQuestion();
            //  applyAnswerOnButton();
            CongratPrompt.gameObject.SetActive(true);
        }
        else
        {
            countStep = 0;
            applyQuestion();
            applyAnswerOnButton();
        }

        disableAllButtons(true);
        resetButton();
        defaultAvatarFace();
    }

    public void NextStepping()
    {
        StartCoroutine(nextStep());
    }

    // Update is called once per frame
    void Update() { }

    public void startQuestion()
    {
        //questionLabel.text = QuestionList.instance.question[countQuestion];
    }
}
```